

Semantic Correspondence with Visual Foundation Models

Alberto Ciccomascolo Alessandro Mosca Gabriele Viceconte Matteo Zito
Politecnico di Torino

{s333968, s333956, s349460, s304809}@studenti.polito.it

Abstract

This project tackles semantic correspondence based on Vision Foundation Models, attempting to establish accurate pixel-level alignments between semantically related object parts across images. Leveraging backbone architectures such as DINO and SAM, we propose a multi-stage framework that integrates a training-free baseline, lightweight fine-tuning of the final network layers, and an enhanced prediction strategy based on a window softmax formulation. Extensive experiments on SPair-71k, PF-Pascal, PF-Willow and AP-10K demonstrate consistent performance improvements over standard baselines, while maintaining strong generalization across datasets. The code related to this project is publicly available at https://github.com/AML-Semantic-Correspondence/Semantic_Correspondence.

1. Introduction

Semantic correspondence is a task that consists in finding pixel-level matches between semantically similar parts of objects across different images. In other words, given one keypoint taken from the source image (for example, a point on a TV screen), the goal is to find the corresponding point on the target image. This problem is inherently challenging due to variations in viewpoint, scale, domain shifts, and the discrepancy between geometrically similar but semantically distinct parts (e.g. a window and a PC monitor).

The project described in this report explores how large Vision Foundation Models (VFMs) can be leveraged to solve this task. Recent works, including Tang et al. [8] and Zhang et al. [9, 10], have shown that these models contain rich internal representations that emerge without explicit supervision, providing a reasonable starting point for dense pixel-level semantic matching.

To evaluate the performance of our models, we use the SPair-71k benchmark, a large-scale dataset designed specifically for semantic correspondence. Furthermore, we exploit the prior knowledge of VFMs by employing some pretrained backbones, i.e., DINOv2, DINOv3, and the Seg-

ment Anything Model (SAM).

By comparing these backbones, we aim to establish a training-free baseline and analyze the impact of fine-tuning and sub-pixel refinement with window soft-argmax. The project workflow is divided into several steps:

- **Training-free baseline:** starting from frozen backbones, we run different evaluations on test images, in order to compare the models without a training session.
- **Fine-tuning:** we introduce the fine-tuning session on the train partition of the dataset, followed by evaluations in order to select the best hyperparameters.
- **Prediction with window softmax and sub-pixel refinement:** to improve the prediction accuracy, we update the inference by replacing the standard argmax function with the window soft-argmax.
- **Project extension:** we test on other datasets in order to see if the models can generalize across different domains.

2. Related works

2.1. Models overview

During the main stage of the project, we perform training, inference, and evaluation on three different pretrained models, i.e., DINOv2 [6], DINOv3 [7] and SAM [4]. These models are called **Vision Foundation Models** (VFMs), that is, general-purpose pre-trained models that can be employed to perform a wide range of computer vision tasks, even without the need of any task-specific fine-tuning. Both DINO and SAM exploit the use of **Vision Transformers** (ViTs), which apply transformer architectures to computer vision tasks, allowing them to split an image into fixed-size patches and treat them as tokens. However, DINO and SAM are built for different high-level purposes.

2.2. DINO

DINO [1] (acronym for *DIstillation with NO labels*) is a family of self-supervised models that can learn strong visual representations of the data without the use of explicit human-provided labels. These models act as a sort of "universal feature extractor" which is able to recognize and group features that belong to the same object category hav-

ing no prior knowledge (e.g., a DINO model does not know what a "cat" is, but given a sufficient amount of images containing cats, it can group together a set of common features that it recognizes as belonging to the same type of entity, i.e., the cat). This ability is based on a specific self-supervised mechanism, called *self-distillation*. In fact, the DINO models are made of two identical architectures with different goals: a teacher and a student. The mechanism operates by passing different "crops" (i.e., views) of the same image through both networks; while the teacher receives global views to establish a stable target representation, the student is tasked with predicting that same output from local, smaller patches. Over time, the student is updated thanks to the loss function while the teacher indirectly "absorbs" the student knowledge in order to stay always one step ahead of it.

2.3. SAM

The Segment Anything Model (SAM) [4] has a different goal and is designed to generate object masks from flexible prompts. In particular, the idea is to produce a valid mask for any prompt, even when the prompt is ambiguous or could refer to multiple objects. To handle such cases, SAM is ambiguity-aware, which means that, rather than collapsing multiple interpretations into a single and averaged mask, it predicts several plausible masks that correspond to different plausible interpretations of the object indicated by the prompt (e.g., clicking on a person's shirt will give masks for their shirt, upper body, and full body.) SAM operates by first encoding an image into a dense feature map using a ViT based image encoder. The user's prompt is then processed by a prompt encoder, which converts clicks, bounding boxes, masks, or text into embeddings that can interact with the image features. A mask decoder combines the prompt and image features to produce one or more segmentation masks that highlight which pixels belong to the object indicated by the prompt.

2.4. Models versions

For each of the previous models, we rely on the base version. In particular, we consider the following.

- DINOv2 ViT-B/14 (with registers): backbone made of 12 transformer layers with approximately 86 million parameters. This version of DINOv2, like all the others, was trained on a collection of more than 142 million unlabeled images. The patch size of the internal vision transformers is set to 14 pixels.
- DINOv3 ViT-B/16: backbone made of 12 transformer layers with approximately 86 million parameters. In this case, the training set is much larger than the one used for DINOv2 and reaches 1.7 billion unlabeled images. The patch size of the internal vision transformers is set to 16 pixels.

- SAM ViT-B: backbone made of 12 transformer layers with approximately 94 million parameters trained on 11 million images and an average of 100 segmentation masks per image. The patch size of the internal vision transformers is set to 16 pixels.

2.5. Overview of datasets

In this study, we consider several datasets designed not only for semantic correspondence. Most of the project simulations (i.e., the ones of the first three tasks) were performed on SPair-71k, which is the starting point for both the training-free baseline and the fine-tuned models.

2.5.1. SPair-71k

SPair-71k [5] is a large-scale benchmark designed to perform semantic correspondence between objects contained in image pairs. It contains 70,958 image pairs across a variety of object categories, including animals, vehicles, and everyday objects. Each pair consists of a source image and a target image, which are to be provided as input to the network. Both images are annotated with a set of keypoints and a bounding box that highlights the object on which the keypoints are located. The bounding box of the target image is particularly relevant because it is used to evaluate the PCK after each prediction. Of course, for the task to make sense, the target image must belong to the same category as the source image, or at least there must be semantic continuity between them (e.g. a cruise ship and a canoe are both boats even if they have a different structure).

SPair-71k is equipped with a "small" version of the data, which contains only a fraction of the image pairs present in the original "large" split, and each partition is divided into specific subsets for training, validation, and testing.

Lastly, the benchmark presents several challenges, such as variations in scale, viewpoint, and intra-class appearance, which provide a sort of "difficulty score" to each image pair and that are meant to test the ability of a model to adapt to different conditions related to the way in which the objects are represented and positioned in the image.

Figure 1 shows an example of an image pair from SPair-71k.

2.5.2. PF-PASCAL and PF-WILLOW

For the last task of the project, we also take into account two other datasets, PF-PASCAL [2] and PF-WILLOW [2], introduced in the following:

PF-PASCAL: a benchmark dataset specifically designed for semantic correspondence that consists of 1351 image pairs belonging to 20 different categories with different distributions (we have 6 image pairs for the "sheep" category, 140 image pairs for "bus", and 67 image pairs on average for each category). The number of keypoints per image is also variable and goes from a minimum of 3 to a maximum of 30. Unlike SPair-71k, PF-PASCAL contains images with

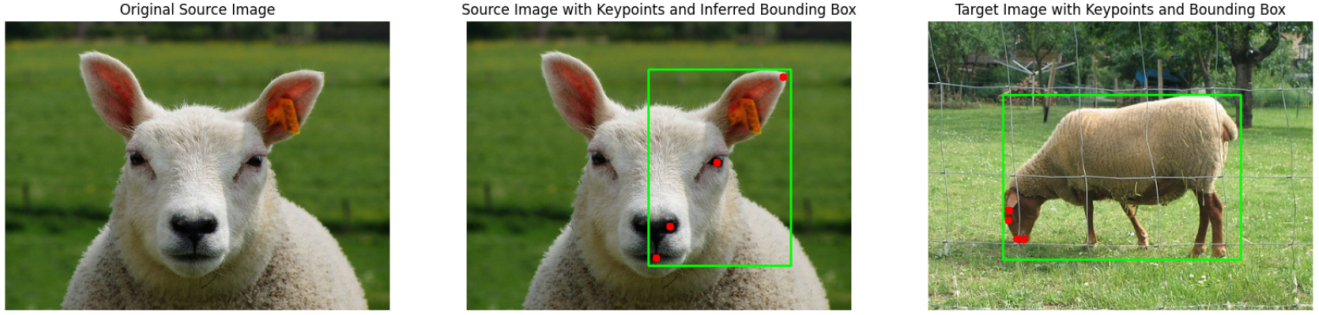


Figure 1. An example of how the SPair-71k dataset is structured. Each data item consists of a pair of images, i.e., the *source image* and the *target image*. Each image comes with a group of keypoints and a bounding box.

similar viewpoints, similar scales and fully-visible objects (i.e., no occlusion), which makes it easier to deal with, especially when using a model that was fine-tuned on SPair-71k.

PF-WILLOW: this dataset is yet another benchmark used for semantic correspondence. It contains 900 image pairs belonging to 5 different categories ("Face", "Car", "Motorbike", "Duck" and "WineBottle") and defines exactly 10 keypoints for each image. PF-WILLOW is even less complex than PF-PASCAL due to its low intra-class variability, low visual complexity, and fixed number of keypoints. Because of these characteristics, PF-WILLOW is often used to perform a "sanity check" (i.e., it is used as a baseline to check whether a model is able to perform semantic correspondence reasonably well before moving on to more complex datasets). Unlike SPair-71k and PF-PASCAL, this last dataset does not explicitly define the target bounding box, so we derive it by using the furthest keypoints for box dimensions, as shown in Figure 2.

It is important to notice that all the previous datasets (i.e., SPair-71k, PF-PASCAL and PF-WILLOW) are based only on **sparse keypoints**, meaning that the objects on which we perform semantic correspondence are defined by a set of semantically meaningful sub-parts expressed using coordinates of specific points (e.g. we compare two images of cats by considering a set of keypoints which correspond to meaningful anatomic parts like the eyes, the tail and the ears). An alternative approach is the so-called **dense correspondence**, in which the comparison between two objects is performed by looking at the entire set of pixels that compose them, providing a continuous mapping rather than a discrete mapping based on a set of keypoints.

2.5.3. AP-10K

AP-10K [3] is a large visual benchmark for mammals pose estimation. It's composed of 10.015 images from 23 animal families and 54 species. The images are manually annotated and refer to a standard set of 17 keypoints that highlight a series of body parts that are common to the anatomical

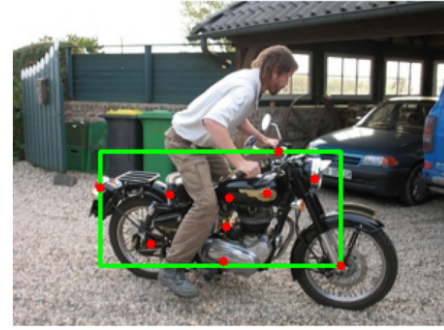


Figure 2. An example coming from PF-WILLOW showing a target image with the related bounding box whose dimensions are set according to the furthest keypoints.

structure of all mammals. This particular dataset has two main aspects that make it more complex for semantic correspondence tasks, that is:

- Although the object categories inside AP-10K are only animals (mammals in particular), the dataset still has a high internal variability given by the great variety of different poses, viewpoints and levels of occlusion, which can alter the semantic interpretation of some keypoints. Moreover, AP-10K also requires cross-species or cross-family correspondences (it may happen that we have to compare a cat to a lion).
- In some image pairs, it may happen that a target image contains more than one animal of the same species and this can create some ambiguities related to which one of those animals should be the target of the matching. This issue is solved by considering all the animals of the same species (inside the same image) as different instances and providing to the model only a crop of one of those animals instead of the entire target image.

Note: because of the extreme variability in terms of visibility (i.e., the level of occlusion), some image pairs have no keypoints in common because there are no keypoints that are visible in both images. In this work, we chose

to skip these image pairs and perform semantic correspondence only between those image pairs with at least one visible keypoint in common.

3. Training-free baseline

3.1. Evaluation metric

In this project, we evaluate the performance of the models using the PCK@ t accuracy metric. The general formula for PCK for a single image is the following:

$$PCK@t = \frac{\text{Number of Correct Keypoints}}{\text{Total Number of Keypoints}},$$

where each keypoint is considered correctly predicted if $d(k_P, k_{GT}) \leq t \cdot L$, that is, if the distance between the predicted keypoint and the related ground truth is less (or equal) than a certain measure that is proportional to the threshold t , with $0 \leq t \leq 1$. To assess the quality of the generated keypoints, we set the value of L as $\max(H_{T_{BBox}}, W_{T_{BBox}})$, i.e., the largest dimension of the target bounding box T_{BBox} .

The thresholds that were considered for models performance assessment are 0.05, 0.1 and 0.2, that is, 5%, 10% and 20% of $\max(H_{T_{BBox}}, W_{T_{BBox}})$.

Moreover, in the following tables, we report the performance results with respect to two different evaluation approaches:

- **PCK@ t per keypoint:** it computes the percentage related to the mean of the correct predictions over the total number of keypoints, separated by threshold. This corresponds to the following formula:

$$(PCK@t)_{\text{keypoint}} = \frac{\text{Training Keypoints Correctly Predicted}}{\text{Total Number of Training Keypoints}}$$

This means that this metric is image-independent and it's not sensitive to the different keypoints distribution among the training image pairs.

- **PCK@ t per image:** it computes the mean percentage related to the mean of the correct predictions over the total number of keypoints per image, separated by threshold. This corresponds to the following formula:

$$(PCK@t)_{\text{image}} = \frac{\sum_{i=0}^{N_I} (PCK@t)_i}{N_I},$$

where N_I is the total number of images and $(PCK@t)_i$ is the PCK with threshold t related to the i -th image.

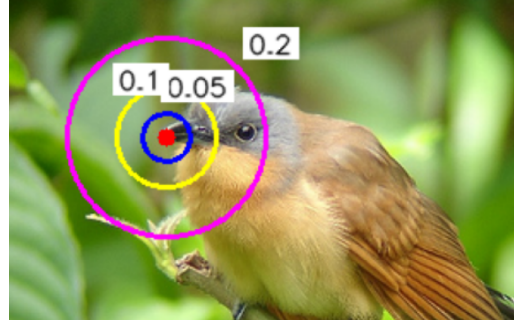


Figure 3. An example showing a zoomed portion of a target image from SPair-71k. The image contains a keypoint corresponding to the beak of a bird along with the areas of tolerance related to the three thresholds used to evaluate the PCK.

3.2. Approach

We start the inference task by freezing the entire model backbones and by using the pretrained weights to perform semantic correspondence without any task-specific fine tuning, in order to test the ability of the VFMs to adapt to a generic computer vision task without being specifically trained for it. In particular, we perform a simple inference by providing the source and the target image to the VFM, which returns two multi-dimensional embeddings, one for each image. The type and dimensions of the embeddings depend on the specific backbone that is being considered. However, the common aspect of these embeddings is the fact that they encode numerical information for each single patch of the related image. This information is often called *descriptor* and consists of a vector of the form $D \in \mathbb{R}^{E \times 1}$, where E is the embedding dimension, which depends on the model version (e.g. we have $E = 768$ for the base version of each model). At this point, for each source keypoint, we perform a cosine similarity between the source descriptor corresponding to that keypoint and the target embedding. This produces the similarity map, which is a numerical representation of how close each target patch is, with respect to the source patch containing the keypoint. Then, we apply an argmax function on the similarity map, obtaining the index of the target patch with the highest matching score. Finally, we use this index to map the related patch back in the target image, extracting the corresponding target keypoint coordinates (x_k, y_k) .

Table 1 shows the PCK results (both per image and per keypoint) related to the three frozen pretrained models, evaluated on the large test split of SPair-71k [5]. These results confirm that large pretrained models naturally retain some information that can be used to perform semantic correspondence without explicit supervision or a training specifically designed for that kind of task.

However, there is a clear distance between the performance on SAM and the one on DINOv2 and DINOv3,

Model	Layers	PCK Analysis per Image			PCK Analysis per Keypoint		
		PCK@0.05	PCK@0.1	PCK@0.2	PCK@0.05	PCK@0.1	PCK@0.2
DINOv2	1	50.40 (+14.47)	65.02 (+12.59)	77.51 (+9.09)	53.77 (+15.46)	68.66 (+13.43)	80.85 (+9.47)
	2	53.94 (+18.01)	68.36 (+15.93)	79.79 (+11.37)	57.53 (+19.22)	72.14 (+16.91)	83.28 (+11.90)
	3	52.18 (+16.25)	65.35 (+12.92)	76.31 (+7.89)	56.43 (+18.12)	70.11 (+14.88)	80.81 (+9.43)
DINOv3	1	46.92 (+14.57)	64.00 (+15.71)	77.08 (+14.24)	51.04 (+14.30)	68.62 (+14.96)	81.43 (+12.99)
	2	51.85 (+19.50)	68.60 (+20.31)	80.40 (+17.56)	56.28 (+19.54)	73.19 (+19.53)	84.48 (+16.04)
	3	54.31 (+21.96)	70.16 (+21.87)	81.52 (+18.68)	58.41 (+21.67)	74.59 (+20.93)	85.32 (+16.88)
SAM	1	23.35 (+11.25)	33.42 (+15.14)	47.20 (+13.69)	27.81 (+15.83)	38.88 (+20.93)	53.38 (+20.28)
	2	22.80 (+10.70)	32.26 (+19.98)	45.47 (+11.96)	27.49 (+15.51)	37.97 (+20.02)	51.98 (+18.88)
	3	20.90 (+8.80)	29.66 (+11.38)	42.46 (+8.95)	25.57 (+13.59)	35.25 (+17.30)	48.73 (+15.63)

Table 2. Comparative PCK results for DINOv2, DINOv3 and SAM on the SPair-71k large test split with different levels of fine-tuning. Each value shows the increment with respect to the related performance without fine-tuning.

which is related to the different philosophy of these VFMs. In particular, SAM produces segmentation masks and so it focuses on establishing whether a portion of an image belongs to an object or not, regardless of the category to which that object belongs.

Analysis	Model	PCK@.05	PCK@.1	PCK@.2
per image	DINOv2	35.93	52.43	68.42
	DINOv3	32.35	48.29	62.84
	SAM	12.10	18.28	33.51
per keypoint	DINOv2	38.31	55.23	71.38
	DINOv3	36.74	53.66	68.44
	SAM	11.98	17.95	33.10

Table 1. PCK results on the large test split of SPair-71k obtained using the free-baseline backbones.

In contrast, DINOv2 and DINOv3 focus on learning discriminative semantic descriptors, which gives them the ability to group features based on conceptual similarity. So, when SAM needs to match a source keypoint on a target image, it may select a target keypoint that has a similar local geometry but that is part of an object that belongs to a conceptually different category (e.g., it could match the ear of a cat with a leaf because it has no specific knowledge about the overall difference of those objects as separate categories).

4. Light fine-tuning

4.1. Loss function

The first requirement to train a model (or, in this case, the last part of a model) is to define an appropriate loss function. For this task, we employ a **cross-entropy loss** with **label smoothing**. This loss function is applied on a scaled version of the similarity map where each score is divided by a fac-

tor τ (defined as a hyperparameter). Then the cross-entropy internally calculates a softmax function on the scaled similarity map, turning it from a score matrix into a probability distribution. With this implementation, the loss value associated to a single keypoint of a single image is calculated as follows:

$$L = - \left[q_y \log \left(\frac{e^{z_y}}{\sum_{j=1}^K e^{z_j}} \right) + \sum_{i \neq y} q_i \cdot \log \left(\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \right) \right],$$

where z are the scaled similarity scores, y is the flattened index of the patch containing the correct target keypoint and K is the total number of patches in the target feature map (which depends on both the model and the image size).

The standard approach (without label smoothing) uses "hard labels", so the probability q_y assigned to the ground truth is 1.0 and all the other patches have probability $q_i = 0$. With label smoothing, instead, we assign a probability $q_y = (1.0 - \alpha + \frac{\alpha}{K})$ to the ground truth and the remaining probability is equally distributed among the other patches, so $q_i = \frac{\alpha}{K}$. For our fine-tuning we set the hyperparameter α to 0.1. Label smoothing improves the generalization capabilities of the model and delays overfitting when fine-tuning involves more than one layer.

At last, the overall loss for an entire batch of images is calculated as follows:

$$L_{\text{batch}} = \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=1}^{M_i} L_{i,j} \right),$$

where N is the batch size, M is the number of keypoints in the i -th image pair and $L_{i,j}$ is the loss value of the j -th keypoint of the i -th image.

4.2. Hyperparameter fine-tuning

Efficient training requires us to use the correct hyperparameters, which, in our case, were: learning rate, weight decay,

batch size, number of epochs and the cross-entropy scaling factor τ (also called *temperature*). The fine-tuning strategy that we used was a simple trial-and-error approach that compared the performance results of different values of the hyperparameters and that, in the end, combined the values that obtained the best results.

The following table shows the final values of the hyperparameters that were chosen for the training of all three backbones.

Hyperparameter	Value
Learning rate	$1 \cdot 10^{-5}$
Weight decay	10^{-1}
Number of epochs	1
Batch size	32
Softmax temperature	$5 \cdot 10^{-2}$

During the hyperparameters fine-tuning, we noticed that training the models for a single epoch and with higher values of learning rate and weight decay produced better results if compared with the ones obtained by training on multiple epochs with less restrictive values of those hyperparameters. In particular, in the first case (1 epoch), we experienced a fast convergence that immediately boosted the performance of the models. In fact, starting from the second epoch, the training ended up overfitting and producing worse final results. In contrast, in the second case (10 epochs), we experienced a much slower convergence in which each epoch slightly increased the performance until reaching a loss stagnation around the 8th epoch and producing final results that were slightly worse than the ones obtained in the previous case.

Moreover, regarding the batch size for DINOv2 and DINOv3, we started with a low value (2-4) and progressively increased it to 32 images per batch, in order to stabilize the gradient and avoid high fluctuations of the loss values which always brought premature overfitting when training on more than 1 layer. However, SAM forced us to stop at 8 images per batch due to specific memory limitations.

Note: the number of layers is also to be considered a hyperparameter, which, however, was not fine-tuned since in the next subsection we decided to report the performance results for different numbers of unfrozen layers.

4.3. Results

In Table 2 we report PCK results obtained with all the three backbones and with different levels of fine-tuning, respectively 1, 2 and 3 unfrozen and re-trained layers. These results show a significant increase in performance for all the backbones (from +10% to +20% for each PCK value), although the improvements are not equally distributed among the different models and the different thresholds. For example, DINOv2 has the highest performance improvements for

PCK@0.05, so the model proved to be more effective in localizing features, meaning that it moved some keypoint predictions (which were already correct according to the less restrictive thresholds) closer to the ground truth as its local geometric structure was better recognized and localized. On the other hand, DINOv3 and SAM show a more balanced efficiency between the matching of new objects (related to good results for less restrictive thresholds) and the further improvement in the precision of matches that were already correct (related to good results for more restrictive thresholds). Finally, we can observe that each model has its peak performance with a different number of layers, that is, 2 layers for DINOv2, 3 layers for DINOv3, and 1 layer for SAM. In this context, DINOv3 shows to have a better generalization ability, which delays the overfitting caused by an excessive number of layers (i.e., parameters) to be fine-tuned.

5. Inference with window soft-argmax

We now try to further improve the performance of the models by changing the way in which we perform inference. Up to now, we extracted the best match from the target image by applying an argmax function on the similarity map, obtaining the index of the patch with the highest similarity score, whose center was then used to calculate the coordinates of the predicted keypoint. However, the simple use of the argmax forces us to assign keypoints only with respect to the center of the patches, making the matching patch-discretized. Moreover, the fact that we only focus on the highest similarity patch (and not also on the neighboring ones) makes the inference prone to noisy predictions (e.g. we may match a keypoint on a patch that corresponds to an isolated spike on the similarity map while the actual target keypoint is somewhere else).

These limitations are the reason for which we now employ the window soft-argmax instead of the simple argmax, as proposed by Zhang et al. [10]. With this new approach, we define a specific window size S_W and, once we have the patch with the highest matching, we extract the portion of the similarity map with dimension $S_W \times S_W$ centered in that patch. In other words, we consider both the patch resulting from argmax and its neighboring patches. Then we divide the values of the extracted window by a temperature value ρ and apply the softmax function, turning the similarity scores into probabilities. Finally, we calculate the coordinates of the keypoint prediction by taking a weighted average of the coordinates of all the entries of the window (expressed with respect to the similarity map from which the window was extracted). This weighted average is the key solution to the limitations of the argmax; the prediction coordinates are not tied to the center of the patches anymore because they can cover any value in the continuous space between the patches and the noisy spikes are less relevant

since we may also take into account the weight of dense regions of lower similarity (if present inside the window).

Of course, both the window size S_W and the temperature ρ are hyperparameters and must be chosen accordingly. After a brief session of hyperparameter fine-tuning, we selected $S_W = 5$ for the window size and $\rho = 5 \cdot 10^{-2}$ for the soft-argmax temperature.

In Table 3 we can observe the improvements related to the best performing fine-tuned models, combined with the window-based soft-argmax.

Analysis	Model	PCK@.05	PCK@.1	PCK@.2
per image	DINOv2	+3.11	+1.47	+0.68
	DINOv3	+3.17	+2.01	+0.54
	SAM	+0.76	+0.34	+0.15
per keypoint	DINOv2	+3.34	+1.58	+0.59
	DINOv3	+3.50	+1.97	+0.51
	SAM	+0.82	+0.37	+0.20

Table 3. PCK increments obtained by switching from the simple argmax to the window soft-argmax. In both cases the inference is performed on the large test split of SPair-71k and using the best fine-tuned versions of DINOv2 (2 layers), DINOv3 (3 layers) and SAM (1 layer).

These results show that the use of the window-based soft-argmax provides a consistent performance boost across all the models and all the thresholds. In particular, the most notable improvement is related to the strictest threshold (i.e., 0.05), whose best improvement reaches +3.50% for DINOv3. In fact, lower thresholds are the ones that suffer the most from the quantization error of the argmax, because the "precision" of the prediction is limited by the centers of the patches. In this sense, the soft-argmax provides reasonable and coherent results.

6. Project Extension

In this last section of the project we consider the best models fine-tuned on SPair-71k and employ them on new datasets, in order to evaluate their generalization capabilities. The goal is to establish whether the fine-tuned models just "memorized" the SPair-71k training data or whether they are able to deal with data coming from a different source. As already explained in section 2.5.2, we now use two datasets designed for semantic correspondence (PF-PASCAL and PF-WILLOW) and one designed for pose estimation (AP-10K).

6.1. PF-PASCAL

In Table 4 we can see the results related to PF-PASCAL. Each entry reports the performance of the pre-trained base-

lines on the new dataset along with the increment given by the use of the best fine-tuned model for each backbone (again, 2 unfrozen layers for DINOv2, 3 unfrozen layers for DINOv3 and 1 unfrozen layer for SAM). In both cases, the inference is made using the window-based soft-argmax. The table shows a general improvement in performances, although this improvement is not homogeneous and has some exceptions. In particular:

- The fine-tuned version of DINOv2 improves the performances in a way that is coherent to what was done on SPair-71k (see the increments in Table 1), i.e. the improvements become more relevant as the threshold is more restrictive.
- The fine-tuned version of DINOv3 has a consistent improvement for PCK@0.05 but it shows a poor generalization ability for the other two thresholds, meaning that this model is good only at generalizing high-precision matching, while the high-level structure detection is too tied to SPair-71k.
- The fine-tuned version of SAM is the one that has the most homogeneous improvements and it proves to be the model with the best and most balanced generalization ability.

Overall, the absolute performance on PF-PASCAL is higher than the one on SPair-71k, as a consequence of its "simplicity" in terms of internal variability and objects representation.

6.2. PF-WILLOW

In Table 6 we can find the performance evaluated on PF-WILLOW. These results are a confirmation of what we already observed for PF-PASCAL, that is: (1) the fine-tuned version of DINOv2 focuses on high-precision matching, but it also performs reasonably well with higher thresholds; (2) the fine-tuned version of DINOv3 is efficient, mainly on high-precision matching, but it lacks the ability to generalize the high-level structure detection; (3) SAM is quite consistent among the different thresholds.

Model	PCK@0.05	PCK@0.1	PCK@0.2
DINOv2	38.10 (+12.16)	64.28 (+10.55)	87.66 (+4.67)
DINOv3	41.63 (+12.99)	72.54 (+2.29)	91.51 (+2.19)
SAM	22.41 (+8.61)	36.96 (+11.02)	53.38 (+12.04)

Table 6. Comparative PCK results for DINOv2, DINOv3 and SAM on PF-WILLOW with the best fine-tuned versions of these models. Each entry shows the performance of the free-baseline backbone and the related increment obtained with the fine-tuned model.

Finally, we can observe that, on PF-WILLOW and for PCK@0.05, both the pre-trained baselines and the fine-

Model	PCK@T analysis per image			PCK@T analysis per keypoint		
	PCK@0.05	PCK@0.1	PCK@0.2	PCK@0.05	PCK@0.1	PCK@0.2
DINOv2	52.23 (+11.96)	72.59 (+8.59)	86.95 (+5.74)	54.64 (+10.64)	74.97 (+7.22)	88.88 (+4.63)
DINOv3	55.53 (+13.08)	89.91 (+4.87)	93.43 (+0.13)	57.14 (+11.9)	82.05 (+3.58)	94.09 (+0.12)
SAM	21.30 (+10.88)	33.19 (+12.39)	49.09 (+13.04)	22.29 (+10.32)	34.46 (+11.64)	49.91 (+12.54)

Table 4. Comparative PCK results for DINOv2, DINOv3 and SAM on PF-PASCAL with the best fine-tuned versions of these models. Each entry shows the performance of the free-baseline backbone and the related increment obtained with the fine-tuned model.

Model	PCK analysis per image			PCK analysis per keypoint		
	PCK@0.05	PCK@0.1	PCK@0.2	PCK@0.05	PCK@0.1	PCK@0.2
DINOv2	31.24 (+8.06)	46.42 (+5.19)	61.72 (+2.43)	30.02 (+7.09)	46.43 (+4.71)	62.48 (+2.29)
DINOv3	27.02 (+11.97)	42.88 (+9.00)	57.53 (+6.03)	25.39 (+11.49)	42.75 (+8.70)	57.95 (+6.20)
SAM	9.24 (+4.92)	15.93 (+6.43)	29.92 (+7.86)	8.95 (+4.78)	16.11 (+6.61)	30.46 (+8.22)

Table 5. Comparative PCK results for DINOv2, DINOv3 and SAM on AP-10K with the best fine-tuned versions of these models. Each entry shows the performance of the free-baseline backbone and the related increment obtained with the fine-tuned model.

tuned models of DINOv2 and DINOv3 produce worse results with respect to the ones on PF-PASCAL and Spair-71k. This is likely related to a specific characteristic of the PF-WILLOW dataset, which contains many images in which the objects appear distant. So, since the calculation of the threshold radius is related to the bounding box of the object in which the keypoints are located, if the object is not in the foreground, it will look smaller and its bounding box will be smaller as well, meaning that the related tolerance radius will be much shorter, even for less restrictive thresholds. This aspect, along with the fact that we define the bounding box by looking at the keypoints and not at the whole object, makes PF-WILLOW a dataset that requires more precision in the keypoint matching. This phenomenon, however, does not involve SAM, which exploits its segmentation strategy that focuses on object boundaries rather than internal textures (like DINO), and that allows it to lose less information, in case of far objects.

Note: In Table 6 we reported the results without expressing the type of analysis, as in PF-WILLOW each image pair has the same number of keypoints, so the PCKs per image and the PCKs per keypoint have the exact same values.

6.3. AP-10K

The results related to the last extra dataset are shown in Table 5. In this case we can see a notable improvement given by the models fine-tuning and, overall, the trend of the previous datasets is confirmed as the highest performance increment for DINOv2/DINOv3 is related to PCK@0.05 while SAM shows an improvement that is more shifted towards less restrictive thresholds.

7. Conclusions

This work investigated the use of some state-of-the art Vision Foundation Models to face a semantic correspondence

task. The training-free baselines confirmed the fact that semantic correspondence naturally emerges from general-purpose large pretrained models without any preliminary task-specific training. Moreover, the fine-tuning of the three backbones on SPair-71k further improved the task performance and resulted in a set of models that showed to have discrete and coherent generalization ability on other datasets designed for either semantic correspondence or pose estimation. The use of VFMs with different approaches (universal feature extraction for DINO and object segmentation for SAM) proved that the ability in recognizing local geometric structures and internal textures is more effective than being able to distinguish the boundaries of the entire objects. Finally, we saw how sub-pixel refinement via window soft-argmax was able to improve the prediction accuracy, having a notable impact on the task performance.

References

- [1] Mathilde Caron, Hugo Touvron, et al. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021. 1
- [2] Bumsub Ham et al. Proposal flow: Semantic correspondences from object proposals. In *IEEE*, 2017. 2
- [3] Hang Yu et al. Ap-10k: A benchmark for animal pose estimation in the wild. In *arXiv:2108.12617v2*, 2021. 3
- [4] Alexander Kirillov, Eric Mintun, et al. Segment anything (sam). In *ICCV*, 2023. 1, 2
- [5] Juhong Min, Jongmin Lee, Jean Luo, and Minsu Cho. Spair-71k: A large-scale benchmark for semantic correspondence. In *ICCV*, 2019. 2, 4
- [6] Maxime Oquab, Timothée Darcet, et al. Dinov2: Learning robust visual features without supervision. In *CVPR*, 2023. 1
- [7] Oriane Simeoni et al. Dinov3. *Preprint*, 2025. 1
- [8] Luming Tang, Akansha Meng, Bo Zhao, et al. Emergent correspondence from image diffusion. In *NIPS*, 2023. 1

- [9] Junyi Zhang, Yi Huang, et al. A tale of two features: Stable diffusion complements dino for zero-shot semantic correspondence. In *NIPS*, 2023. [1](#)
- [10] Junyi Zhang et al. Telling left from right – identifying geometry-aware semantic correspondence. In *CVPR*, 2024. [1](#), [6](#)